.NET Integration (existing API in ~10 minutes)

1. **Reference one package** (transitively pulls the rest): `dotnet add <YourApi>.csproj reference <path>/src/Pixous.AI.AspNetCore/Pixous.AI.AspNetCore.csproj` (or install the packed `Pixous.AI.*` NuGets from your feed).
2. **Register + map** in `Program.cs` (order: after auth/CORS; `UseRateLimiter` only when `RateLimit:Enabled`): ``csharp builder.Services.AddAiSuite(builder.Configuration); // builder.Services.AddAuthentication(...); // your JWT/OIDC — unchanged app.UseCors("FrontendOnly"); app.UseAuthentication(); app.UseAuthorization(); if (app.Configuration.GetValue<bool>("AiSuite:RateLimit:Enabled")) app.UseRateLimiter(); app.MapAiSuiteEndpoints(); // /api/ai/chat, /chat/stream, /translation/*, /providers, /info, /selection ``
3. **Copy the `AiSuite` section** from `samples/SampleApi/appsettings.json`. Start with `ChatProvider: KnowledgeBase` / `TranslationProvider: Echo` — the API works immediately with zero keys.
4. **Store secrets** (User Secrets → env → Key Vault); flip `Security:RequireAuthorization` once your auth scheme exists.
5. **Verify**: `GET /api/ai/info` → providers; `POST /api/ai/chat {"message":"hi"}` → reply. Full calls: `samples/sample-requests.http`.

Plugging enterprise pieces (no core edits)

Need	How
SQL/Mongo/Redis history	Implement `IConversationStore` (3 methods), register BEFORE `AddAiSuite` (yours wins); send `sessionId`
Your auth (JWT/OIDC/existing)	Nothing in the library changes — add your scheme, flip `RequireAuthorization`
New AI/translation vendor	`IChatProvider`/`ITranslationProvider` adapter + `AddSingleton` + config (providers.md)
Tenant-specific prompts/limits	Per-environment config; or wrap providers per tenant in your project
Admin settings API	Call `IChatService`/`ITranslationService`/`AiSuiteRuntimeSelection` from your controllers